University *of Ljubljana*
Faculty *of Computer and Information Science*

Text-to-ADQL: Query Generation for the Gaia DR3 Database

Guillem Masdemont Serra, Pietro Sestito, and Plabon Shaha

Abstract

Accessing the ESA Gaia Data Release 3 (DR3) catalogue requires expertise in the Astronomical Data Query Language (ADQL) and careful query optimisation to avoid server timeouts, creating a significant barrier for non-specialist users. To address this, we present a natural language pipeline that translates human-made questions into valid, safe, and executable ADQL queries, building on decomposition and self-correction strategies from the text-to-SQL literature. The pipeline features a two-path architecture governed by a shared cost judge that ensures queries are safe to execute before submission. We evaluate the architecture on a custom dataset and we achieve a 55% rate of structurally correct or near-correct queries. We present the results through an interactive HTML interface.

Advisor: Aleš Žagar

1. Introduction

The Gaia space telescope, launched in 2013 by the European Space Agency (ESA), aims to produce the most detailed map of the Milky Way ever assembled, cataloguing approximately one billion stars with astrometric, photometric, and spectroscopic measurements [1]. The most recent public release, Gaia Data Release 3 (DR3, 2022), contains 1.8 billion sources described by 258 catalogue fields [2], occupying 10 TB.

Access to the data is mediated through the Gaia Archive, which exposes a Table Access Protocol (TAP) endpoint that accepts queries written in the Astronomical Data Query Language (ADQL[1]) [3]. While ADQL is well-suited to the domain, executing queries programmatically within this framework poses several barriers. The *astroquery.gaia* Python library is prone to connection timeouts, and on the ARNES cluster, stable *TAP access* requires establishing an SSH tunnel to bypass network restrictions. Beyond this, queries against a 1.8-billion-row table are inherently delicate, and errors occur even for tightly constrained queries.

Our project builds a natural language pipeline that translates human-written questions into valid, safe, and executable ADQL queries for Gaia. Our problem domain is a specific instance of *text-to-SQL* [4], with several complicating factors absent from standard benchmarks:

1. There is no existing dataset of paired natural language questions and ADQL queries for Gaia.
2. Complex questions often require decomposition into multiple queries (e.g. "*show me the HR diagram and the colour histogram of the Pleiades*" requires joining two independent queries).
3. The Gaia TAP server enforces strict execution time limits, so generated queries must be optimised to target very specific table scans.

To address these challenges, we propose a pipeline that combines the query decomposition strategy of DIN-SQL [4] with the multi-stage architecture of MAC-SQL [5]. The result is a two-path design with a shared deterministic cost judge that vets every query before execution, with the goal to ensure we meet the TAP server's execution time constraints. The simple path handles simple queries through a validate-build loop, while the complex path decomposes complex queries into a structured plan and processes each step through separate validate-build loops. As a concrete example, Figure 1 shows the pipeline handling *"Provide me the brightest stars"*: the router classifies this as a simple query with a `stellar_population` intent, and the builder produces a sky-wide ADQL filtering on apparent magnitude:

```
SELECT TOP 1000
  source_id, ra, dec, phot_g_mean_mag, bp_rp,
  pmra, pmdec, parallax
FROM gaiadr3.gaia_source
WHERE phot_g_mean_mag IS NOT NULL
  AND MOD(random_index, 1000) = 0
ORDER BY phot_g_mean_mag ASC
```

Since this is a sky-wide query without a spatial cone, the cost judge injects a row-sampling predicate to avoid scanning

[1] a dialect of SQL extended with geometric predicates for sky-coordinate filtering (e.g. `CONTAINS`, `CIRCLE`, `POINT`).

the full 1.8 billion rows, rating the query as moderate and returning 1,000 sources in under 5 seconds. The results are displayed through an HTML interface as interactive visualisations.

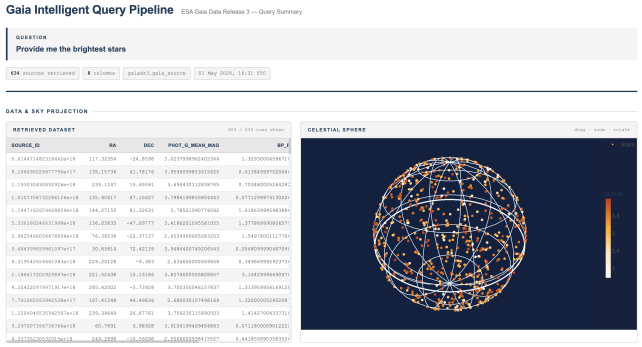

Figure 1. Pipeline output for *"Provide me the brightest stars."* The simple path issues a sky-wide ADQL query ordered by apparent magnitude, returning 1,000 sources displayed on a celestial sphere projection.

We evaluate our pipexfline on a custom-created dataset using LLMs such as Claude-opus-4-5 and Mistral served through Snowflake Cortex AI. We observe that the proposed architecture constructs the exact intended query with a 28% precision, and produces a query of equivalent intent in up to 54% of cases.

2. Method

This section describes the design of our natural language pipeline for Gaia. Our problem is a specific instance of text-to-SQL, a mature research area with established benchmarks such as WikiSQL and BIRD [6]. Adapting it to ADQL introduces additional challenges absent from these benchmarks: domain-specific spatial predicates (Contains, CIRCLE, POINT) and strict server-side execution constraints. We address these through a two-path architecture where every generated ADQL string passes through a shared cost judge before execution, guaranteeing uniform safety across both paths.

2.1 Data and Schema

Given the scale and complexity of the Gaia DR3 schema, a key design decision is restricting the pipeline to a tractable subset of columns and query types. Queries execute live in the Gaia DR3 TAP service with no local copy of the data, imposing two hard constraints: generated ADQL must be syntactically correct before dispatch, and must be cost-aware, as the server enforces a wall-clock limit of approximately 60 seconds per asynchronous job. The primary table is `gaiadr3.gaia_source`, which contains 1.8 billion rows and 258 columns. We restrict the pipeline to a curated vocabulary of 18 columns covering astrometry[2], photometry[3], stellar parameters from the GSP-Phot pipeline[4], and quality flags[5]. The pipeline supports eight query intents, shown in Table 1.

Intent	Cone req.	JOIN	Sky-wide
`cone_search`	✓	–	–
`color_histogram`	✓	–	–
`variability_search`	✓	–	–
`hr_diagram`	–	–	✓
`stellar_population`	–	–	✓
`velocity_computation`	–	–	✓
`internal_crossmatch`	–	✓	✓
`nearest_neighbours`	–	–	✓

Table 1. Supported query intents and their key ADQL constraints.

In the following sections we describe the full architecture as shown in Figure 2, consisting of a router that classifies queries, a simple pipeline path, and a complex pipeline path.

2.2 Router

When a question is fed into the pipeline, it first passes through an LLM router built on `Qwen2.5-7B` served via vLLM, following the separation of concerns of MAC-SQL [5]. The router is a single-turn classifier: it receives the user query and returns a two-field JSON object (complexity: simple | complex; reason: short string). The prompt includes ten few-shot examples covering the key ambiguous cases: queries with spatial filters or crossmatches that resolve to a single ADQL, and queries with conjunctions ("*and also*", "*compare*") that require multiple queries. The router defaults to *simple* when in doubt, reducing unnecessary use of the complex path. Sampling is deterministic with `temperature=0`. The query is then routed to either the simple or complex path.

2.3 Simple Pipeline Path

The simple path handles single-intent queries through a four-stage loop with up to three retries.

Stage 1 — LLM Parser. The parser maps the input question to a fixed JSON schema with eight fields: the query intent (one of the eight types in Table 1), a spatial anchor (right ascension, declination, and radius), a column selection list, WHERE filters as typed range or equality constraints, a join table, and a row limit. Following the schema-pruning principle of CHESS [7], the system prompt exposes only the valid column set alongside intent descriptions and ten worked examples, mitigating column hallucination.

Stage 2 — JSON Validator. A deterministic function validates the parsed JSON and enforces a correct format, checking that:

- The chosen intent is in the supported set of Table 1 and all mandatory fields are filled with non-null values.

[2] `ra`, `dec`, `parallax`, `pmra`, `pmdec`, `radial_velocity`
[3] `phot_g_mean_mag`, `phot_bp_mean_mag`, `phot_rp...`
[4] `teff_gspphot`, `logg_gspphot`, `mh_gspphot`
[5] `phot_variable_flag`, `ruwe`

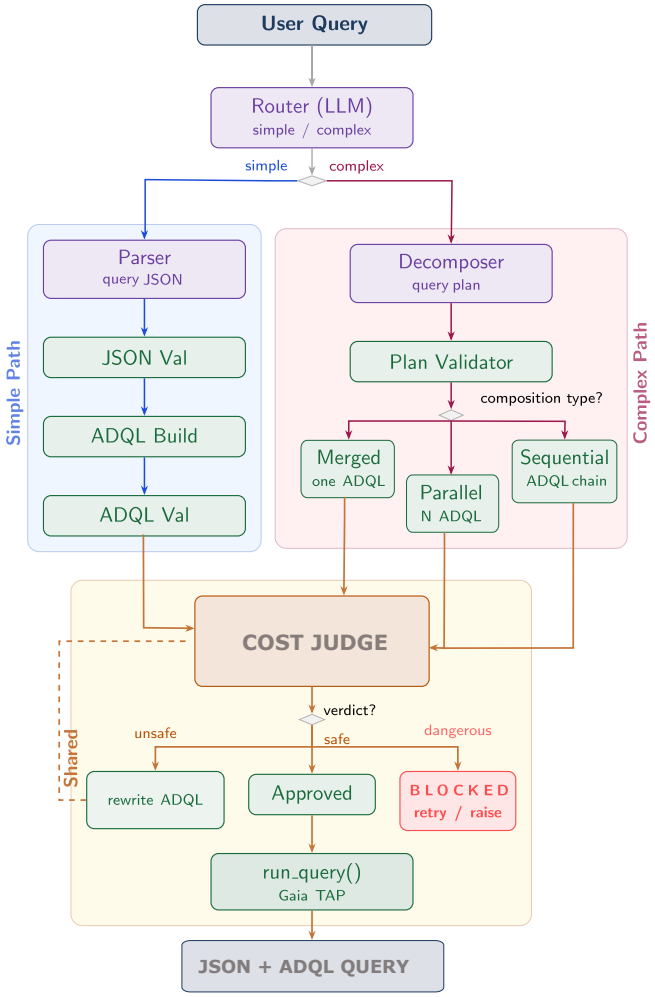

Figure 2. Overview of the proposed pipeline. A router classifies each query as simple or complex and dispatches it to the corresponding path. Both paths converge on a shared cost judge before executing the final ADQL query against the Gaia TAP server.

- All column names are in `VALID_COLUMNS` and spatial fields are numeric and within valid ranges ($0 \leq \text{ra} \leq 360$, $-90 \leq \text{dec} \leq 90$, radius $\leq 10°$).

On failure, a structured correction prompt containing the invalid JSON, a numbered error list, and the original query is fed back to the parser for up to three retries, following the self-correction loop design of DIN-SQL [4] and CHASE-SQL [8].

Stage 3 — ADQL Builder. Once the JSON is validated, a deterministic builder constructs the ADQL by applying an intent-specific template. Every query includes a fixed set of base columns; cone-required intents use a spatial containment predicate; for queries involving stellar parameters such as temperature or metallicity, results are filtered to only include stars where those measurements are available, since they are not recorded for every source in the catalogue; and all intents order results by a random index for unbiased sampling, except for nearest neighbours which orders by parallax in descending order.

Stage 4 — ADQL Validator. Once the ADQL is built, a final deterministic check verifies that it contains no literal `None` (indicating an unresolved coordinate), a `TOP` clause (to bound the result set), and a `WHERE` clause (to avoid a full-table scan). Any failure re-enters the correction loop with a structured error prompt.

2.4 Complex Path

In case the router detects a complex query, it triggers the complex path, which handles requests that require combining multiple ADQL queries. The path operates in three stages: planning, validation, and execution.

Decomposer (planning) The first stage converts the natural language query into a structured plan via an LLM call with a dedicated system prompt and five few-shot examples. The plan describes *what* needs to be done, declaring a composition type (*merged*, *parallel*, or *sequential*), a shared spatial region, and the list of sub-queries to execute, but does not generate any ADQL at this stage.

Plan Validator Before any query is built, a deterministic validator checks that the plan is coherent: merged plans must have at least two compatible intents and a defined spatial region; sequential plans must have no circular dependencies; and any intent requiring a spatial cone must have a region specified. Only once the plan passes validation does the pipeline proceed to execution.

Composition modes (executor) The validated plan is then executed according to its composition type, producing the final ADQL queries. **Merged**: two or more analyses on the same region are combined into a single ADQL query. **Parallel**: independent queries are dispatched through the simple path separately, each returning its own dataset. **Sequential**: dependent queries where the results of step n are injected into step $n+1$ as a filter on source identifiers, forming a chain of nested queries.

Outer retry loop. If any step fails, the orchestrator builds a plan-level correction prompt and re-enters the decomposer for up to two more retries.

2.5 Shared Cost Judge

After the query is produced, either by the simple or complex path, it is not executed immediately. Given the strict execution time limits of the Gaia TAP server, every generated ADQL first passes through a shared cost judge that determines whether the query is safe to execute as-is, requires optimisation, or should be rejected. We adapt the scoring of CHASE-SQL [8], so we score queries by execution safety rather than answer quality. The judge comprises three components:

Fast pre-check (rule-based, no LLM): immediately flags as dangerous any query missing a `WHERE` clause (would scan

all 1.8 billion rows) or a `TOP` clause, or with `TOP` exceeding 100,000.

LLM judge: queries that pass the pre-check are scored on a 0–100 scale by a separate LLM call. The judge returns a verdict (cheap/moderate/expensive/dangerous), a score, and concrete rewrite suggestions. Scores 0–25 are safe to run as-is; 26–75 trigger the auto-optimiser; 76–100 re-enter the correction loop.

Auto-optimiser: applies a sequence of deterministc rewrites to make the query progressively more constrained and easy to retrieve. Specifically we use *MOD(random_index, N) = 0* for sky-wide queries without a cone, with *N* ranging from 100 to 100000 based on score. This enables uniform sampling over a reduced subset.

Once the cost judge pass is finished, the output is a JSON record containing the ADQL query, the plan and the per-step ADQL strings. The json can then be used to query Gaia.

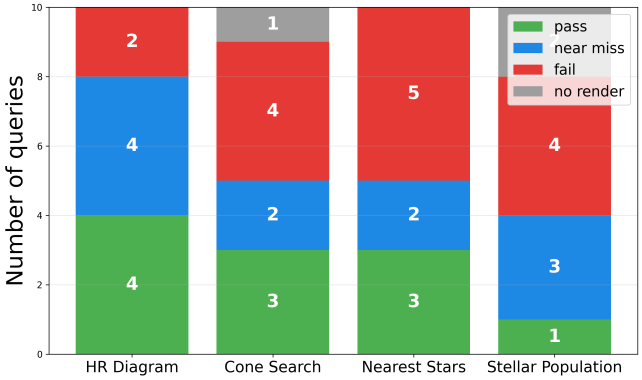

Figure 3. Pipeline performance per intent on the 40-query evaluation set. Green bars indicate fully valid predictions, blue bars indicate near-misses (a single failing check), red bars indicate failures with two or more failing checks, and grey bars indicate queries the pipeline could not render.

3. Evaluation

3.1 Dataset

Since no text-to-ADQL benchmark exists for Gaia DR3, we generated an evaluation corpus using Snowflake AI Cortex and calling mistral-large and claude-opus-4-5. The generation prompt is constrained to use the 18 valid columns, the eight intent templates, and a single WHERE filter per query, ensuring structural compatibility with the pipeline output. We restrict the evaluation to the four intents that admit simple single-filter queries. For position-based queries, we ground the dataset on 10 well-known stellar objects, ensuring that coordinates are real and verifiable rather than hallucinated. We generate 10 question-ADQL pairs per intent for a total of 40 examples.

3.2 Results

Each generated query is evaluated against its reference using five binary checks. A predicted query is considered correct if its result set is contained within the ground truth dataset, meaning it must be at least as restrictive as the reference. This is verified through five checks: the predicted SELECT must cover at least all reference columns; all reference WHERE constraints must be present in the predicted query; the intent must match; each numeric constraint must be satisfied at least as tightly; and for spatial queries, the cone centre must lie within 0.5° of the reference and the predicted radius must not exceed it. A query is *valid* only if all five checks pass.

To characterise the failure modes more precisely, we classify each row into four outcomes: *pass* (all five checks satisfied), *near-pass* (exactly one check failed), *fail* (two or more checks failed), and *no_render* (the pipeline returned no ADQL, either through an exception or a 10-second timeout). Results are displayed in Figure 3.

Figure 3 shows the outcome distribution per intent. Overall, 22 out of 40 queries (55%) are at least structurally close to the reference, with 11 fully valid and 11 near-misses, giving a strict pass rate of 28%. The HR diagram intent performs best, as its fixed mandatory clauses leave little room for the model to drift. Cone search and nearest neighbours share the same distribution, with most failures concentrated in the spatial radius check. This is the least critical constraint, since a slightly wrong radius still retrieves a meaningful dataset. The 11 near-misses are the most relevant finding: each of these queries fails on exactly one check, meaning a single targeted correction would make them fully valid. This suggests that improvements to coordinate resolution and intent disambiguation would yield large gains without any architectural changes to the pipeline.

4. Conclusion

We presented a natural language pipeline for querying the Gaia DR3 catalogue and translating human questions into valid and executable ADQL queries. The two-path architecture successfully handles both simple and complex queries while respecting the strict execution constraints of the TAP server. Evaluated on a custom dataset of 40 examples, the pipeline achieves a 28% strict pass rate and a 55% rate of structurally correct or near-correct queries. These results, suggests that improvements on the coordinate resolution and intent disambiguation would yield substantial gains. Future work could include expanding the evaluation dataset, supporting the remaining query intents, and fine-tuning the QWEN2.5-7B model on data to further close the gap between near-miss and fully valid predictions.

References

[1] Gaia Collaboration, T. Prusti, et al. The Gaia mission. *Astronomy & Astrophysics*, 595:A1, 2016.

[2] Gaia Collaboration. Gaia data release 3. *Astronomy & Astrophysics*, 2023. https://archives.esac.esa.int/gaia.

[3] ESA Gaia Archive. Gaia archive visualisation and interface platform (gavip), 2016.

[4] Mohammadreza Pourreza and Davood Rafiei. DIN-SQL: Decomposed in-context learning of text-to-SQL with self-correction. *Advances in Neural Information Processing Systems*, 36, 2023.

[5] Bing Wang, Changyu Ren, Jian Yang, Xinnian Liang, Jiaqi Liang, Lixin Cui, Binyuan Shi, Ruoming Zhang, Yue Zheng, Yanlin Su, Chuanqi Li, and Zhixing Mao. MAC-SQL: Multi-agent collaborative framework for text-to-SQL. *arXiv preprint arXiv:2312.11242*, 2023.

[6] Jinyang Li, Binyuan Hui, Ge Qu, Jiaxi Yang, Binhua Li, Bowen Li, Bailin Wang, Bowen Qin, Rongyu Cao, Ruiying Geng, Nan Huo, Xuanhe Zhou, Chenhao Ma, Guoliang Li, Kevin C. C. Chang, Fei Huang, Reynold Cheng, and Yongbin Li. Can llm already serve as a database interface? a big bench for large-scale database grounded text-to-sqls, 2023.

[7] Shayan Talaei, Mohammadreza Pourreza, Yu-Chen Chang, Azalia Mirhoseini, and Amin Saberi. CHESS: Contextual harnessing for efficient SQL synthesis. *arXiv preprint arXiv:2405.16755*, 2024.

[8] Mohammadreza Pourreza, Masoud Heidari, Ruoxi Shang Kwon, Hailong Zhao, Chenxi Liu, and Pedram Hassanzadeh. CHASE-SQL: Multi-path reasoning and preference optimized candidate selection in text-to-SQL. *arXiv preprint arXiv:2410.01943*, 2024.